GROEIPROJECT
MODULE 3: REFLECTION EN ANNOTATIONS



1. Voorbereiding

1.1. Open het groeiproject en maak daarin een derde module: `m3_reflection`. Pas vervolgens de `build.gradle` aan.

1.2. Kopieer ALLEEN de package **`model`** uit module `1` (met o.a. de basis- en de multiklasse) naar de nieuwe module.

1.3. Creëer in de package `model` een nieuwe klasse die als superklasse voor de basisklasse zal dienen (Dit heet "*generalisatie*"). In ons voorbeeld zal `Dictator` overerven van `Person`. Verplaats een aantal attributen van de basisklasse naar de superklasse, op voorwaarde dat die daar conceptueel beter thuishoren. Verplaats ook de getters/setters die bij deze attributen horen. In ons voorbeeld verschuiven de attributen `name`, `gender` en `birth` van `Dictator` naar `Person`. Zorg ervoor dat het conceptueel zo goed mogelijk klopt.
→ Je kan dit manueel doen, maar gebruik het liefst de menukeuze "*Refactor* > *Extract Superclass*".
→ OPGELET: alle methoden die afhankelijk zijn van de verplaatste attributen verhuizen mee:

- De getters en setters van deze attributen (in ons voorbeeld: `getName, setName, getGender, setGender, getBirth` en `setBirth`)
- Als de `equals` en `hashCode` methode afhankelijk is van een attribuut (attributen) die verhuisd zijn, dan verhuis je deze ook. (in ons voorbeeld is `equals` en `hashCode` gebaseerd op `name`, dus verhuist naar `Person`)
- Als de `compareTo` gebaseerd is op een attribuut (attributen) die verhuisd zijn, dan verhuis je deze ook. (in ons voorbeeld is `compareTo` gebaseerd op `name`, dus verhuist naar `Person`)
- Pas de constructor(s) aan; roep vanuit de subklasse de super-constructor op.
- Pas de `toString` aan; roep liefst vanuit de subklasse `super.toString()` aan.

2. Reflection

2.1. Maak parallel naast de package `model` een nieuwe package `reflection`, met daarin een nieuwe klasse `ReflectionTools`. Maak daarin de volgende static methode:

```
public static void classAnalysis(Class<?> aClass)
```

2.2. De bedoeling is dat je in deze methode **introspection** doet en de klasse analyseert die als parameter binnenkomt (`aClass`). Opgelet: je moet alles kunnen doen met deze generieke `aClass`-parameter; je mag dus **nergens** een cast doen naar een andere klasse!

2.3. Maak in deze methode een oplijsting van de volgende informatie over de klasse en druk af in een keurig overzicht (baseer je op de voorbeeldafdruk 2.6 op de volgende pagina). Overgeërfde informatie hoeft **niet** getoond te worden.

a. volledige klassenaam ("fully qualified name")
b. naam van de superklasse
c. naam van de package
d. gebruikte interfaces

e. aanwezige constructors (TIP: `constructor.toGenericString()`)

f. namen van de private attributen met tussen haakjes hun type. (Voorbeeld: `birth(LocalDate)`)

g. getters, setters en andere methoden

2.4. Maak een main-klasse `Demo_3` waarin je de methode `classAnalysis` oproept met **3 parameters**:

- de superklasse (zie hierboven 1.3 ; in ons voorbeeld is dat: `Person.class`)
- de basisklasse (in ons voorbeeld: `Dictator.class`)
- de multiklasse (in ons voorbeeld: `Dictators.class`)

2.5. Maak nu in de header van de methode `classAnalysis` gebruik van een **varargs**-parameter, zodat je meerdere argumenten kunt meegeven met de oproep ervan. De werking en afdruk moet hetzelfde blijven.

In ons voorbeeld wordt de oproep in de main dan:

```
ReflectionTools.classAnalysis(Person.class, Dictator.class, Dictators.class);
```

2.6. Gewenste output (liefst even mooi geformatteerd):

```
Analyse van de klasse: Person
=============================================
Fully qualified name   : be.kdg.dictatorproject.model.Person
Naam van de superklasse: Object
Naam van de package    : package be.kdg.dictatorproject.model
Interfaces: Comparable
Constructors:
       public
be.kdg.dictatorproject.model.Person(java.lang.String,be.kdg.dictatorproject.model.Gender,ja
va.time.LocalDate)
attributen     : name(String) gender(Gender) birth(LocalDate)
getters        : getBirth getGender getName
setters        : setGender setBirth setName
andere methoden: equals toString hashCode compareTo compareTo

Analyse van de klasse: Dictator
=============================================
Fully qualified name   : be.kdg.dictatorproject.model.Dictator
Naam van de superklasse: Person
Naam van de package    : package be.kdg.dictatorproject.model
Interfaces:
Constructors:
       public
be.kdg.dictatorproject.model.Dictator(java.lang.String,be.kdg.dictatorproject.model.Gender,
java.time.LocalDate,java.lang.String,java.lang.String,int,double,double)
       public be.kdg.dictatorproject.model.Dictator()
attributen     : country(String) regime(String) duration(int) cruelty(double)
              victims(double)
getters        : getCountry getRegime getDuration getVictims getCruelty
setters        : setRegime setCruelty setCountry setDuration setVictims
andere methoden: toString

Analyse van de klasse: Dictators
=============================================
Fully qualified name   : be.kdg.dictatorproject.model.Dictators
Naam van de superklasse: Object
Naam van de package    : package be.kdg.dictatorproject.model
Interfaces:
Constructors:
       public be.kdg.dictatorproject.model.Dictators()
attributen     : dictatorSet(Set)
getters        : getSize
setters        :
andere methoden: add remove toString search sortedOnName sortedOnBirth sortedOnVictims
```



3. Annotations

3.1. Maak onder de package `reflection` een nieuwe annotation met de naam `@CanRun`.
Deze annotation zal gebruikt worden *boven een methode* en moet *at runtime* beschikbaar zijn. Het is een zogenaamde "single-value" annotation, met één attribuut `value` van het type String. Stel als default-value "*dummy*" in.

3.2. Gebruik deze annotation in je basisklasse en de superklasse daarvan (in ons voorbeeld `Dictator` en `Person`) boven alle **setters die een String als parameter hebben**. Geef via de annotation een specifieke String-value mee (of gebruik de dummy-default) Voorbeelden:

```
@CanRun("België")
public void setCountry(String country) { //... }

@CanRun("Democratie")
public void setRegime(String regime) { //... }

@CanRun //zonder value-parameter, dus "dummy"
public void setName(String name) { //... }
```

3.3. Schrijf in de klasse `ReflectionTools` een nieuwe static methode `runAnnotated` met volgende signatuur:

```
public static Object runAnnotated (Class<?> aClass)
```

3.4. In deze methode moet je weer vertrekken vanuit de `aClass`-parameter; je mag dus nergens een cast doen naar een andere klasse!

a. Maak een nieuwe instantie aan van deze `aClass`; roep daartoe de default constructor aan. Eventuele exceptions gooi je verder naar de oproepende main.
b. Overloop alle methoden van deze klasse en selecteer daaruit enkel de methoden die geannotteerd zijn via **@CanRun** en die als parameter één String hebben (TIP: `getGenericParameterTypes`)
c. Voer deze methoden uit op het geïnstantieerde object met als parameter de **value** van de annotation.
d. Retourneer het gecreëerde object.

3.5. Roep in de main-klasse `Demo_3` de methode `runAnnotated` op met als parameter je basisklasse (in ons voorbeeld: `Dictator.class`). Druk hier het ontvangen Object af.

Mogelijke output:

```
Aangemaakt object door runAnnotated:
dummy                    (°2021) België              regime: Democratie      0,0 mln doden
```

3.6. Gebruik het terminal-venster van IntelliJ:

- `./gradlew run` → worden alle 3 modules gerund zonder fouten?
- `git status -u` → controleer de "changes not staged for commit" in het rood
- `git add .`
- `git commit -m "Module 3 afgewerkt"` → controleer de gegevens van de committer (= jij)
- `git push` → controleer jouw repository via de link